

1)

		P_1	P_2	P_3	P_4	P_5
22	2	B	B	Running	B	R
37	7	R	R	Swapped	B	Running
47	7	R	R	R	B	Exited.

where B is blocked, R is ~~running~~^{ready}, swapped means it has been swapped and exited means the process has ended.

22, P_1 : Blocked due to I/O read from disk unit 3.

22, P_2 : Blocked due to I/O read from disk unit 2.

22, P_4 : Blocked due to I/O write ~~from~~^{to} disk unit 2.

37, P_2 : Blocked due to I/O write to disk unit 3.

47, P_4 : Blocked due to I/O write to disk unit 3.

Assumptions: P_5 was in memory before $t=22$. After I/O, processes are either in ready / running state.

22, P_3 : is running because at $t=22$, it executes a command.

37, P_5 : is running because at $t=37$ it terminates.

~~2) LRU~~

Belady's anomaly

Belady's anomaly: 16 pages, 1 page fault
Assuming random pages, 16 pages follow a pattern second-chance can perform better than FIFO.

2) LRU - replacement: 2, ~~Yes~~ No

b) FIFO - replacement: 3, ~~Yes~~ Yes

c) Optimal: 1, No

d) Second-chance: ~~4~~ 4, Yes

3) ~~LRU~~: 1, 2, 3, 4, 2, 1, 5, 6, 2, 1, 2, 3, 7, 6, 3, 2, 1, 2, 3, 6

7 frames:

Page fault: 1	1	1	1	0	0	1	1	0	0	0	0	1	0	0	0	0	0	0	0
Page table 1	1	1	1			1	1					1							
		2	2			2	2					2							
			3			3	3					3							
				4		4	4					4							
					5	5	5					5							
							6					6							
												7							

Total page faults = 7.

Since there are 7 frames and 7 different pages, it was obvious that there will be exactly 7 page faults as no replacement is needed. This is same in all methods.

~~page faults:~~

~~page table: LRU, FIFO, OPT~~

In 6 frames:

LRU: 6 + 1 ^{replaces 4} = 7 page faults
FIFO: 6 + 1 ^{replaces 1} + 1 ^{replaces 2} + 1 ^{replaces 3} + 1 ^{replaces 4} = 10
OPT: 6 + 1 ^{replaces 4} = 7

In 5 frames:

LRU: 5 + 1 ^{replaces 3} + 1 ^{replaces 4} + 1 ^{replaces 5} + 0 = 8
FIFO: 5 + 1 ^{replaces 1} + 1 ^{replaces 2} + 1 ^{replaces 3} + 1 ^{replaces 4} + 1 ^{replaces 5} = 10
OPT: 5 + 1 ^{replaces 4} + 1 ^{replaces 5} = 7

In 4 frames:

$$\text{LRU: } 4 + 1^{5 \leftarrow 3} + 1^{6 \leftarrow 4} + 1^{3 \leftarrow 5} + 1^{7 \leftarrow 6} + 1^{6 \leftarrow 1} + 1^{1 \leftarrow 7} = \underline{10}$$

$$\text{FIFO: } 4 + 1^{6 \leftarrow 1} + 1^{6 \leftarrow 2} + 1^{2 \leftarrow 3} + 1^{1 \leftarrow 4} + 1^{7 \leftarrow 5} + 1^{3 \leftarrow 6} + 1^{6 \leftarrow 7} = \underline{11}$$

$$\text{OPT: } 4 + 1^{5 \leftarrow 4} + 1^{6 \leftarrow 5} + 1^{7 \leftarrow 1} + 1^{1 \leftarrow 7} = \underline{8}$$

In 3 frames:

$$\text{LRU: } 3 + 1^{4 \leftarrow 1} + 1^{1 \leftarrow 3} + 1^{5 \leftarrow 4} + 1^{6 \leftarrow 2} + 1^{2 \leftarrow 1} + 1^{1 \leftarrow 5} + 1^{3 \leftarrow 6} + 1^{7 \leftarrow 7} + 1^{6 \leftarrow 2} + 1^{2 \leftarrow 7} + 1^{1 \leftarrow 6} + 1^{6 \leftarrow 1} = \underline{15}$$

$$\text{FIFO: } 3 + 1^{4 \leftarrow 1} + 1^{1 \leftarrow 2} + 1^{5 \leftarrow 3} + 1^{6 \leftarrow 4} + 1^{2 \leftarrow 1} + \cancel{1^{1 \leftarrow 5}} + 1^{1 \leftarrow 5} + 1^{3 \leftarrow 6} + 1^{7 \leftarrow 2} + 1^{6 \leftarrow 1} + 1^{2 \leftarrow 7} + 1^{3 \leftarrow 3} + 1^{3 \leftarrow 6} + 1^{6 \leftarrow 2} = \underline{16}$$

$$\text{OPT: } 3 + 1^{4 \leftarrow 3} + 1^{5 \leftarrow 4} + 1^{6 \leftarrow 5} + 1^{3 \leftarrow 1} + 1^{7 \leftarrow 2} + 1^{2 \leftarrow 7} + 1^{1 \leftarrow 6} + 1^{6 \leftarrow 1} = \underline{11}$$

In 2 frames:

$$\text{LRU: } 2 + 1^{3 \leftarrow 1} + 1^{4 \leftarrow 2} + 1^{2 \leftarrow 3} + 1^{1 \leftarrow 4} + 1^{5 \leftarrow 2} + 1^{6 \leftarrow 1} + 1^{2 \leftarrow 5} + 1^{1 \leftarrow 6} + 1^{3 \leftarrow 1} + 1^{7 \leftarrow 2} + 1^{6 \leftarrow 3} + 1^{3 \leftarrow 7} + 1^{2 \leftarrow 6} + 1^{4 \leftarrow 3} + 1^{3 \leftarrow 1} + \cancel{1^{1 \leftarrow 4}} + 1^{6 \leftarrow 2} = \underline{18}$$

$$\text{FIFO: } 2 + 1^{3 \leftarrow 1} + 1^{4 \leftarrow 2} + 1^{2 \leftarrow 3} + 1^{1 \leftarrow 4} + 1^{7 \leftarrow 2} + 1^{6 \leftarrow 1} + 1^{2 \leftarrow 5} + 1^{1 \leftarrow 6} + 1^{7 \leftarrow 2} + 1^{7 \leftarrow 1} + 1^{6 \leftarrow 3} + 1^{3 \leftarrow 7} + 1^{2 \leftarrow 6} + 1^{1 \leftarrow 3} + 1^{3 \leftarrow 2} + 1^{6 \leftarrow 1} = \underline{18}$$

$$\text{OPT: } 2 + 1^{3 \leftarrow 1} + 1^{4 \leftarrow 3} + 1^{1 \leftarrow 4} + 1^{5 \leftarrow 1} + 1^{6 \leftarrow 5} + 1^{1 \leftarrow 6} + 1^{3 \leftarrow 1} + 1^{7 \leftarrow 2} + 1^{6 \leftarrow 7} + 1^{2 \leftarrow 6} + 1^{4 \leftarrow 3} + 1^{3 \leftarrow 2} + 1^{6 \leftarrow 1} = \underline{15}$$

1 frame:

consecutive same pages

Since no duplicates all will result in 20 page faults.

4) 2 processes, foo and bar are executed concurrently.

1) S, R are shared semaphore (init 1) and $x=0$

2) i)

foo: semwait(S)

bar: semwait(R)

foo: semwait(R) $\rightarrow$ it is waiting

bar: semwait(S) $\rightarrow$ it is waiting

happens, they would ~~not~~ block each other.

b) Only in scenarios where:

foo: semwait(S);

foo: semwait(R);

bar: semwait(R);

foo: ~~wait~~ semsignal(S), semsignal(R), semwait(S), semwait(R).

bar: still waiting

Can a process result in indefinite postponement, however this is

not exactly concurrent since all of foo was executed before bar,

it is considered concurrent then yes it is possible.

5) head = 26

26, 37, 100, 14, 88, 33, 99, 12

a) FCFS: $|26-26| + |37-26| + |100-37| + |14-100| + |88-14| + |33-88| + |99-33| + |12-99|$
 $= 0 + 11 + 63 + 86 + 74 + 55 + 66 + 87$
 $= 442$

b) SSTF: $|26-26| + |33-26| + |37-33| + |37-14| + |12-14| + |12-88| + |99-88| + |100-99|$
 $= 0 + 7 + 4 + 23 + 2 + 76 + 11 + 1 = 124$

c) SCAN: $|26-26| + |33-26| + |37-33| + |88-37| + |99-88| + |100-99|$
 $= 100 - 26 = 74$
 $\downarrow = |100-14| + |14-12| = 100 - 12 = 88$
 $\Rightarrow \text{total} = 162$

d) C-SCAN: $|26-26| + |33-26| + |37-33| + |88-37| + |99-88| + |100-99| = 100 - 26$
 $= 74$

$\downarrow: |10-12| + |12-14| + |100-0| = 100 + 12 + 2 = 114$

total = 188

Assumption 100 is the last point

b) Speed = 15,000 rpm
 $= 250 \text{ rps}$

total byte per sector = 512 B

400 sector in track

$\Rightarrow$ byte per track = $0.5 \times 400 \text{ KB}$
 $= 200 \text{ KB}$

1000 tracks $\Rightarrow$

Seek time = 4ms

To store 1 MB, $\Rightarrow$ ~~5000 KB, we need~~ $5.12 \times 200 \text{ KB}$

$\Rightarrow$ 5.12 tracks or 5 complete tracks and 48 sectors in track 6.

c) transfer time = 5 rotation complete + 1 $\frac{48}{400}$ rotation + 6 seek time

$$= \frac{5 \times \pi}{250} + \frac{48}{400} \times \frac{1}{250} + 6 \times 4 \times 10^{-3} \text{ s}$$

$$= \cancel{0.4485} 0.04448 \text{ s}$$

Assuming we initially need to seek.

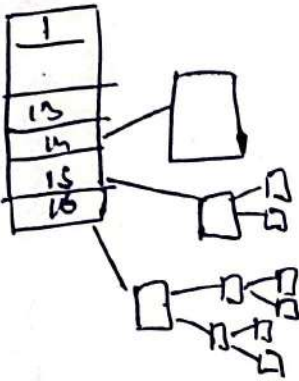
b) Average access time = 4ms

c) Rotational delay is 4ms.

d) Total time to read a sector = $\frac{4}{400} \text{ ms} = 0.01 \text{ ms}$

e) Total time to read a track = 4ms.

7)



~~$13 \times 8 \text{ KB}$ for direct pointers. Since 32 bits, a table can have 2^{32} entries.~~

~~$1 \times 2^{32} \times 8 \text{ KB}$ for 1 indirect pointer~~

~~$1 \times 2^{32} \times 2^{32} \times 8 \text{ KB}$ for double indirect pointer~~

~~$1 \times 2^{32} \times 2^{32} \times 2^{32} \times 8 \text{ KB}$ for triple indirect pointer.~~

~~2^{32}
 2^{16}~~

Total size = ~~$(13 \times 8 + 2^{64} \times 8 + 2^{96} \times 8) \text{ KB}$~~ Assuming each table has 2^{16} entries:
 ~~$\approx (2^7 + 2^{67} + 2^{103}) \times 2^{10}$~~
 ~~$\approx \frac{2^{103}}{2^{103}} \times 2^{10}$~~
 ~~$\approx \frac{2^{113}}{2^{113}}$~~
 ~~$\approx$~~

Direct: ~~$13 \times 8 \text{ KB}$~~
 Indirect: ~~$1 \times 2^4 \times 8 \text{ KB}$~~
 Double indirect: ~~$1 \times 2^4 \times 2^4 \times 8 \text{ KB}$~~
 Triple indirect: ~~$1 \times 2^4 \times 2^4 \times 2^4 \times 8 \text{ KB}$~~

2) Total size: ~~$13 \times 8 + 16 \times 8 + 13 \times 8 + 2^4 \times 8 + 2^8 \times 8 + 2^{12} \times 8$~~

~~$\approx (8(13 + 16 + 256 + 4096))$~~

~~$= 8(4381) \text{ KB}$~~

~~$= 35048 \text{ KB}$~~

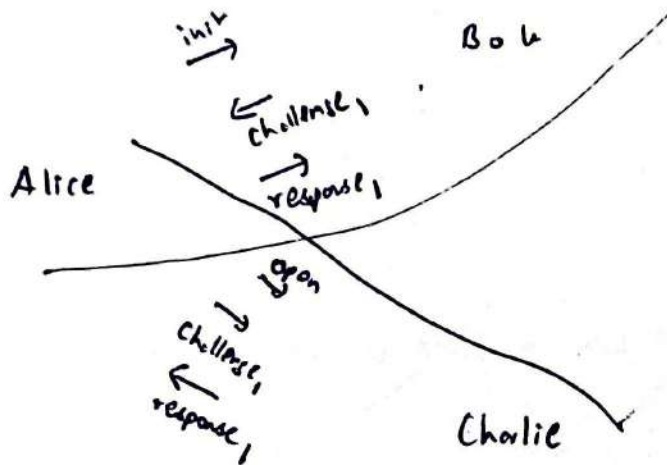
~~$\approx 34.2 \text{ MB}$~~

If table entries $\uparrow$, we can increase our max size.
 For example 16 page entries are 2^8 instead of 2^4 .

Total size = ~~$8(13 + 2^8 + 2^{16} + 2^{24})$~~

~~$\approx 8 \times 2^{25} = 2^{28} \text{ KB or } 2^8 \text{ GB} \approx 256 \text{ GB}$~~

8)



Let a challenge be C_i and its response be r_i .

A simple guard against such an attack could be that authentication can take place only with different (C_i, r_i) pairs.

Since Alice-Charlie auth has already used (C_1, r_1) it cannot be used to authenticate Bob-Alice.

Clearly for this to work, a challenge ^{accept} ~~was~~ ^{via some} could ~~be sent~~ ^{not} multiple responses, ~~it~~

If we have to restrict with 1 challenge - 1 response scheme.

The protocol can add additional info like a timestamp, and it can check if the challenge-response pair has been used before.

Example: Secure MAC, with $m=C$ and $r=b$. (m, t) pair.